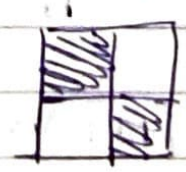
DL GenAI - Week-6

L1 Intro to Generative Models

- Generative Model → type of statistical model that is capable of learning the underlying distribⁿ of a dataset, and generates new, synthetic data samples similar to original data.
  - → models joint $P(X, Y)$ of features & labels
  - → creates new samples
  - → learns the compressed latent space representaⁿ of data.
- capabilities —
  - → data generaⁿ → realistic
  - → understand data → reveal latent struct.
  - → fills in as missing data imputaⁿ
  - → used in anomaly detecⁿ
  - → learns the repreisentaⁿ (low-dimensional)

Eg 2×2 grid of pixels. 0 (B) or 1 (W)
$2^{2\times2} = 2^4 = 16$ possible images.

1 could be  $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

→ Gen. model learns $P(x_{11}, x_{12}, x_{21}, x_{22})$
x_{ij} → pixel value at row i & col j.

→ we can count these occurrences (small no.) and normalize them.

$P(\text{all W}) = 0.3$, $P(\text{diag. B}) = 0.2$ ....

→ for real image, $(256 \times 256 \times 3)$ no of pixels is huge, we can't enumerate them.

→ now based on $P(x_{ij})$ & what is more common, we can start getting samples.

→ $P(x_{11} = 1) \Rightarrow$ top left pixel is white
↳ then 8 possible image, where $x_{11} = 1$.
We can have hypothetical joint probability.
This is marginalization → find the probability of a subset of variables by summing over the others.

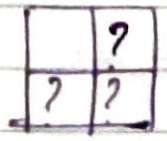
$$P(x_1) = \sum_{x_2} \sum_{x_3} \sum_{x_4} P(x_1, x_2, x_3, x_4)$$

↳ this tells the overall likelihood, now, this is currently irrespective of other pixels.

→ another eg. could be some parts of img. is missing, a gen. model can fill in.

$$P(x_{mis} \mid x_{obs}) = \frac{P(x_{mis}, x_{obs})}{P(x_{obs})}$$

$$= \frac{P(x_{mis}, x_{obs})}{\sum_{x'_{mis}} P(x'_{mis}, x_{obs})}$$



Input

$x_{11} = 1$

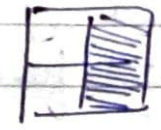
$P(x_{12}, x_{21}, x_{22} \mid x_{11} = 1)$

↳ total 8 combinations

- $P(x_{mis} \mid x_{obs}) = \frac{P(x_{mis}, x_{obs})}{P(x_{11} = 1)}$ ⇒ constant

Generative model: find the $\operatorname{argmax}_{x_{mis}} P(x_{mis}, x_{obs})$

What is (x_{12}, x_{21}, x_{22}) that somehow we maximize $P(1, x_{12}, x_{21}, x_{22})$



(this gets filled in based on learned distribution)

- the bottleneck comes in, when dimensions increase. 2^{1024} ⇒ is astronomically large
  - (32×32) pixels
    - ↳ now this is just a grayscale img.
- Modern models (GANs, VAEs) do not learn full table. They learn a func^n that can approximate this distribution.
- Another possibility,

generative classifier:

$P(x \mid y = c)$ ⇒ use for classification.

$P(y = c \mid x) = \frac{P(x \mid y = c)\, P(y = c)}{P(x)}$

choose class c, that maximizes $P(x \mid y = c)\, P(y = c)$

__/__/__

- Generative classifier → models $P(x|y)$ & $P(y)$
  Can generate data.
  ↳ eg. NB, LDA
- Discriminative classifier → models $P(y|x)$.
  Learns decision boundary
  ↳ Log Reg, SVM, CNN.

→ these are more robust and provide richer information. But they might require a lot of data to learn $P(x|y)$ effectively.

- some eg. OpenAI Sora 2, Google's Imagen 2, Stable Diffusion XL, Meta's Audio Gen.
- 1° goal of goal of generative model will be
  → to learn $p_{data}(x)$ of the real data x.
  ↳ this is a probability distribution.

L2: Intro to Latent Space

- Latent space → lower dimensional representation that captures the essential, underlying structure of high dimensional data.
- High-dim. data → complex, noisy, redundant features, 256×256 ($65,536$)
- Latent space → simplified, meaningful, captures

core concepts.

- 1° goal → dimensionality reduc^n & data interpreta^n.
- PCA.
  - → find orthogonal axes that capture the max. variance of data.
  - → Latent space → subspace spanned by the first k principal components.
  - → used in face detec^n.
- FA (Factor Analysis)
  - → assume obs. data as a linear combina^n of unobserved factors (latent vari) + noise.
  - → used in Psychology
- LDA (Latent Dirichlet Alloca^n)
  - → gen. model for text. Document is a mix of topics & each topic is distribu^n over words.
  - → LS (Latent space) → space of topics
- SVD
  - → decompose a large matrix into smaller matrices

pre-DL Model

- The goal was to analyze existing data. Take high dim. x & find it low-dim. represenṭa^n z.

$$x \longrightarrow \text{Encoder} \longrightarrow z \text{ (Latent space)}$$

unstructured. The focus was on inference (valid), not gener^n.

__/__/__

- Idea is to treat the latent space as a well-behaved probab. distribu^n that we could sample from. (sampling from noise).

Random vector of noise z from Gaussian and use a trained decoder to generate a new data pt. x.

$z \longrightarrow$ decoder (Generator) $\longrightarrow \hat{x}$

$\Downarrow$ estimate this from given data

This was enabled by DL models (VAEs, GANs) which creates a structured, continuous latent space.

- Toy eg. (2×2): Direct probabilistic sampling
  ↳ we sampled from learned discrete distribu^n.
  $P(\text{img.}_A) = 0.3$, $P(\text{img.}_B) = 0.1$
  ↳ this is 3× more likely to be sampled
  ↳ the noise comes from the randomness of this direct sampling process
- DL models (GANs) ⇒ latent space sampling
  ↳ for real imgs., we can't store $P(x)$! They learn a mapping func^n from latent space to complex data space. The noise z is pt. sampled from latent distribu^n. Finally, we get the image. The "noise" brings variety.

__/__/__

- Another eg: rolling a loaded die. ⇒ $P(x)$.
  The randomness is in the roll. In deep model, random set of DNA instructions (x), with diff. combinaⁿ, we get diff. creatures.
- Modern models use NN to learn highly-complex real data.
- Pre-DL → models $P(x)$, enumerate probab. They struggle with high-dim. data like images.
- Need for LS → Real imgs. exist on a very low-dim. manifold within the vast pixel space. Most of them are just noise. We learn the mapping from z to the data manifold.
- Imagine the space of all images. Only tiny fracⁿ is useful. We pick a random pt. from latent space z. Train a complex funcⁿ to transform into meaningful imag.
- This idea was bought by AE: (Autoenc.) by learning represenⁿ ⟹ VAEs.
- GANs said we can mapping w/o modelling $p(x)$.
- Diffusion models emerged as another approach using structured noise.
- The paradigm has shifted from data analysis to data synthesis. Now, we will learn how to model this distributions.

L3 Types of Generative Models

1. Explicitly density models → define & learn funcⁿ for $p(x)$.

__/__/__

2. Implicit Density models → they learn a process to sample from $p(x)$ w/o ever defining the func^n itself.

- Explicit density models.
  → evaluates $p(x)$
  → Tractable → density $p(x)$ computed directly
    1. AR → model joint probab. as product of conditionals
    2. Normalize Flows → complex func^n using invertible func^n
  → Approximate → $p(x)$ is intractable, so optimize an approx^n.
    1. VAEs → maximize the lower bound on likelihood.
    2. Diffusion → add noise to data. Used in SOTA images
       ↓↓
       score-based models.

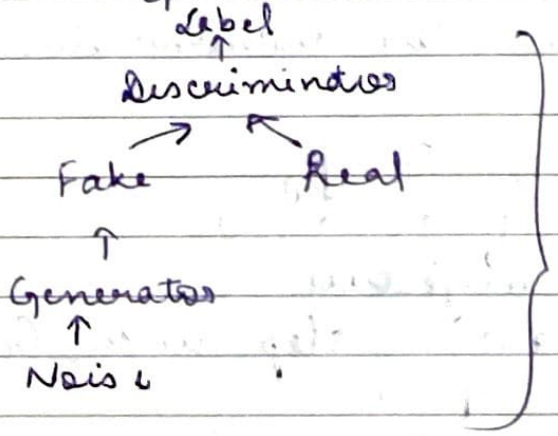
- Implicit density models
  → not define $p(x)$
  → GANs → 2 players → Generator & Discriminator
    ↓ create fake data to fool disc.
    ↓ distinguish real from fake data.

    ⇓
    different types are possible.

- VAE might result in blurry outputs.

L4 Intro to GANs (Generative Adversarial Networks)

- Idea → make NNs compete against each other in the hope that this competition will push them to excel.
  Game → Artist & Critic
  (Generator) (Discriminator)
  Game ends when artist is good if critic gets fool half of the time.
- GAN → Generator + discriminator



1. D → the classifier → learn to distinguish btw. real & fake data.

$$\max_D \left(\underbrace{E_{x \sim p(data)(x)} [\log D(x)]}_{\text{correctly label real as real}} + \underbrace{E_{z \sim p_z(z)} [\log (1 - D(G(z)))]}_{\text{classify fake as fake}} \right)$$

2. G → Creator → generates images.

$$\min_G \left(E_{z \sim p_z(z)} [\log (1 - \underbrace{D(G(z))}_{\text{close to 0}})] \right)$$

↳ pushes D's output for fake images towards 1.

- GAN Training Objective

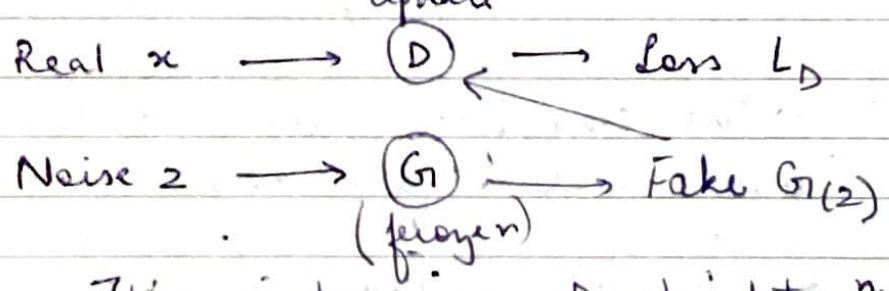
↳ Adversarial Learning (locked in a direct comp.)

$$\min_{G} \max_{D} V(D, G) = E_{x \sim p_{data}(x)}[\log D(x)] + E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

The goal is to find the nash equilibrium.

- Phase-1 ⇒ Train the discriminator

1. $x \sim p_{data}$, $z \sim p_z$
   Generate fakes $G(z)$
2. max the objective
3. gradient ascent step with G frozen



. Thus, improving D at detec^n.

- Phase-2 ⇒ Train generator

1. $z \sim p_z$, generate $G(z)$
2. min the objective
3. gradient descent step

Repeat this until bit convergence of minimax game.

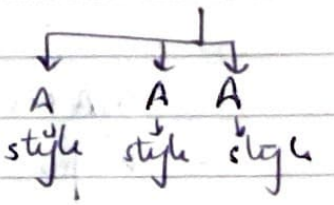
AdaIN → Adaptive Instance Norm.

__/__/__

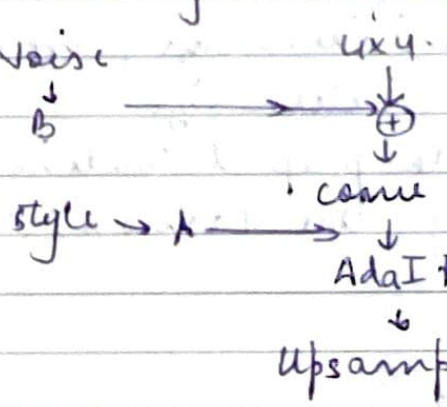
↗ deep convolutated GANs.

- DCGANs → for large images.
  1. Replace pool by strided conv.
  2. use batch norm.
  3. remove FC layers
  4. use Relu in G, except in output use tanh. Use leaky Relu in D.
- Style GANs → SOTA arch. improved image quality by using style transfer.

1. maps network $z \in Z \rightarrow FC \rightarrow \dots FC \rightarrow w \in W$
   (reduce correlations)
   w → multiple style vectors.



2. synthesis network (AdaIN)
   and gets stochastic variation



- training GANs is very difficult & unstable.
  → Mode Collapse → img. becomes less diverse
  G forgets other classes.
  WGAN → aims to reduce mode collapses
  → non-convergence
  → Vanishing gradients
  → HPT high-sensitivity

- WGAN (Wasserstein GAN)

→ measures wasserstein distance btw. real & generated distribu

- Minibatch D → if diversity reduces it flags those images. ∴, G creates variety.
- Tero progressive growing of GANs → first generate small images (4×4). Then move further.

L5 Fashion MNIST: Overview & Play with data

- GANs using PyTorch
- import the essential libraries
- create a dir to save generated samples
- HPT selec" & configura"
  latent_size, hidden_size, image_size, num_epochs, batch_size
- move more to data prep & loading
  ↳ we are using the Fashion MNIST dataset

L6 Model Implementa"

- generator module
  Input → random noise vector
  Hidden → FC with leaky Relu
  Output → image space vector.
- similarly define the discriminator module

Leaky Relu → max(0.2x, x).

__/__/__

- perform the forward pass
- discriminator must not be either too strong or too weak.
  → output will be a sigmoid for the binary classifier (fake / real)
- now define the training configuration.
- optimizer → Adam (Adaptive Moment Estimation)
  loss → BCE loss
  → should be sep. for D & G. As 1 will be frozen while optimizing the other one.
- define the utility func^n that helps in training → norm, reset-grad, etc.

L7 Adversarial Training Loop

- GAN Training loop as discussed in theory.
  → goal → generate increasingly realistic images
  → first train D, then G.
  The iter^n continues until Nash Eqⁿ is reached
- log all the results for monitoring purposes.

18 Model Eval^n

- evaluate the model & plot the train / test losses and results.